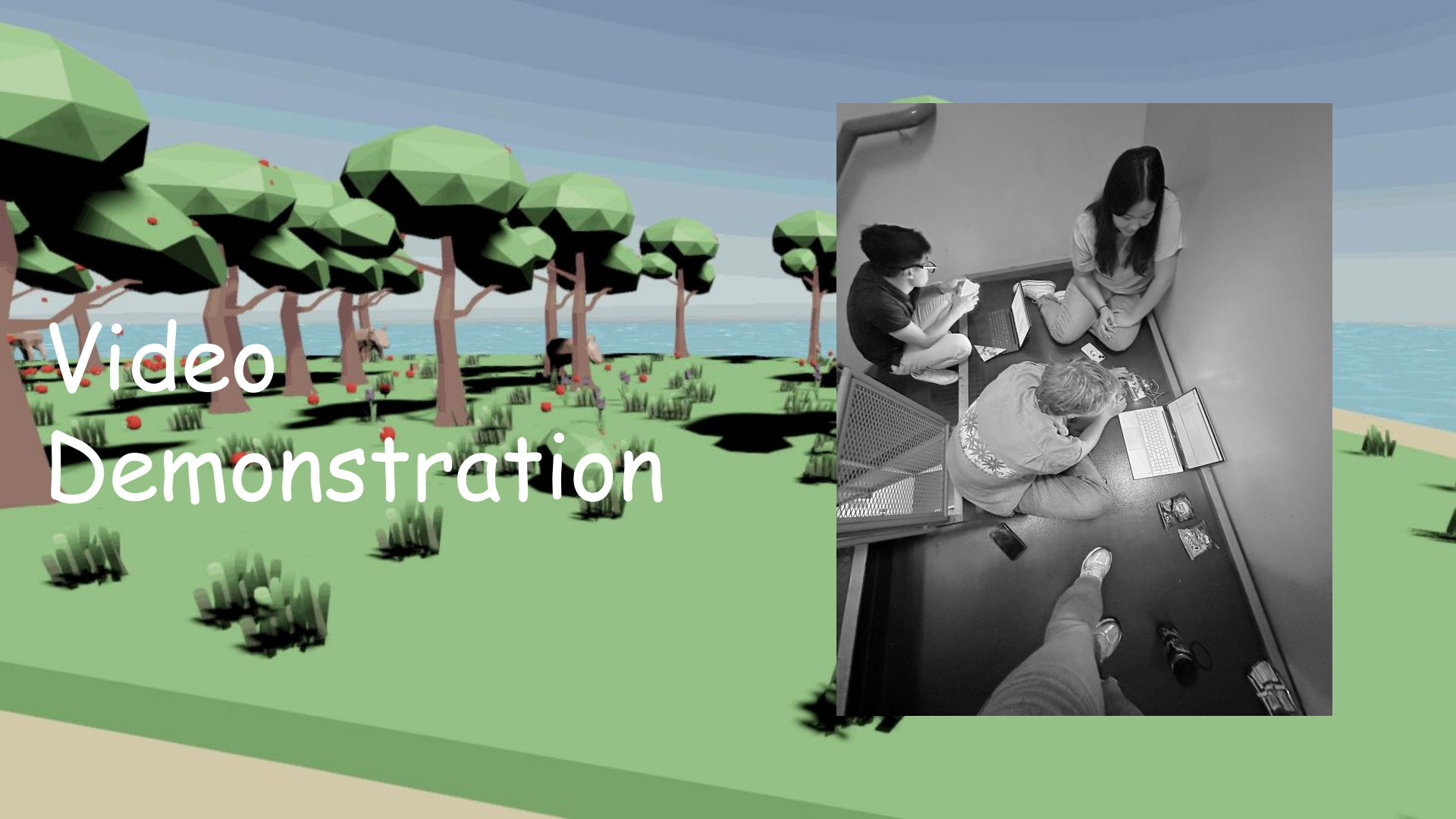


Morse Code Translator

Skye Pattee
Lilian Doan
Brian Ngo
Isaac Hoyt

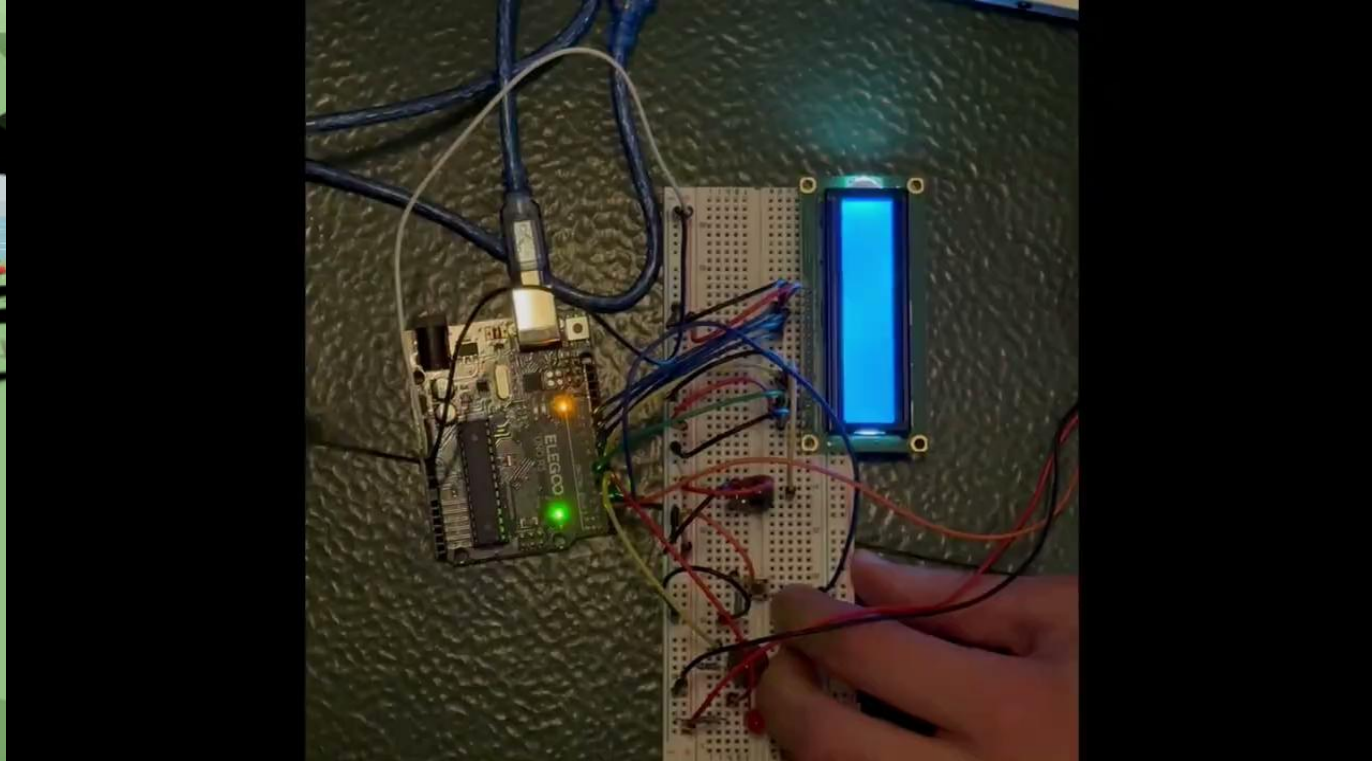




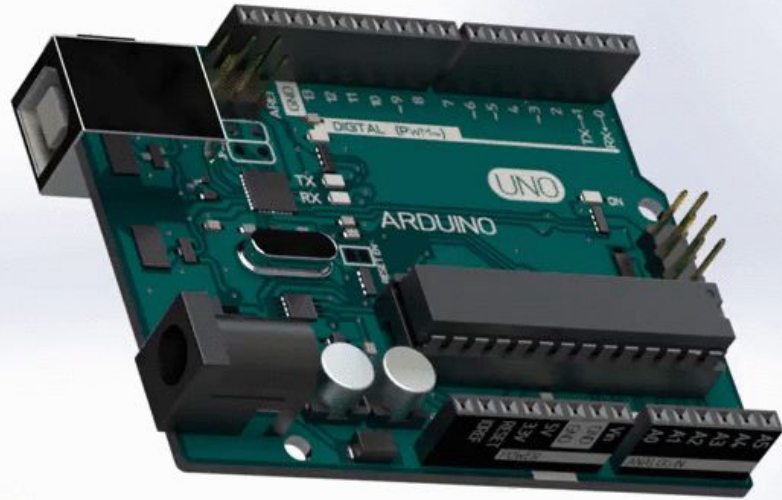
Video Demonstration



Video Demonstration



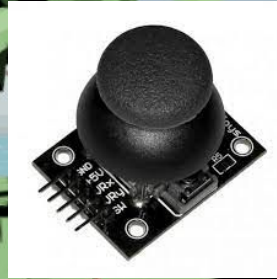
Hardware Components Setup



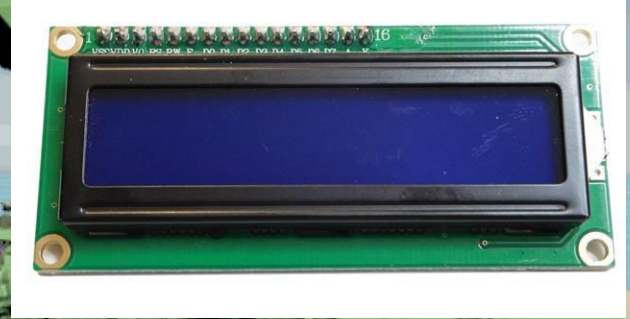
buzzer



joystick



LCD1602
(Liquid Crystal Display)



LED



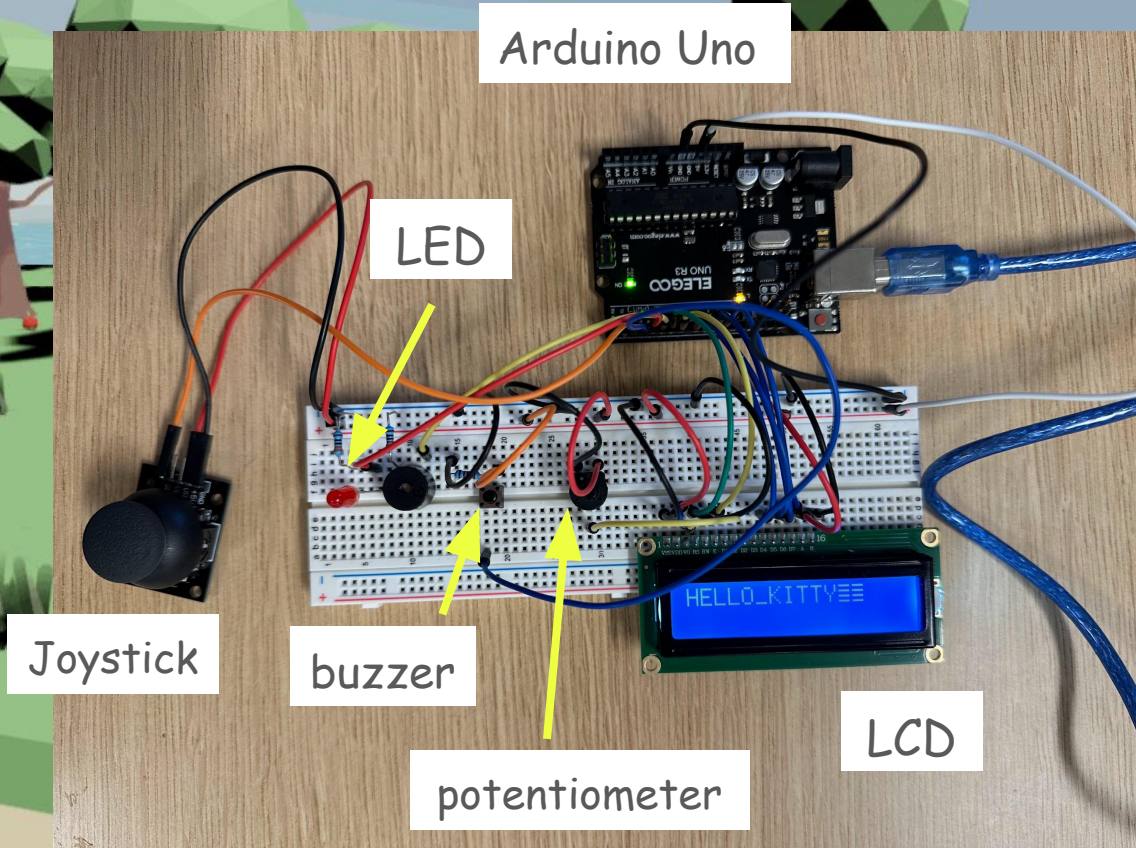
button



potentiometer



Final Form



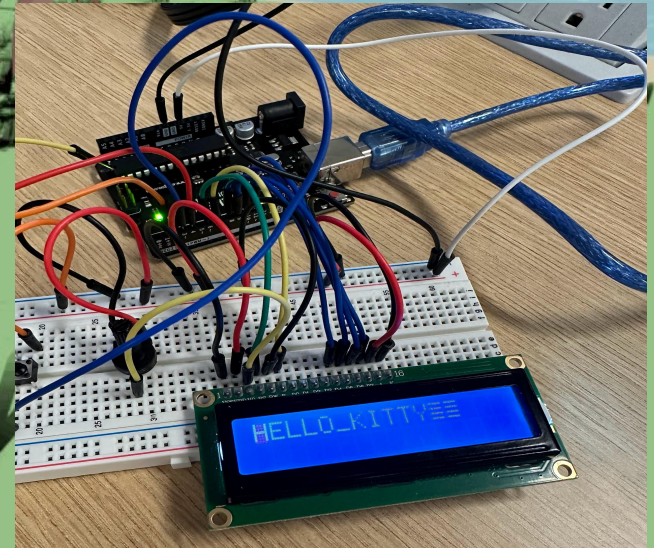
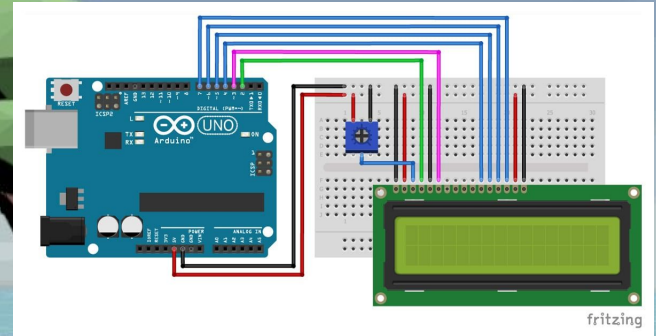
Hardware Struggles

- LCD Display

```
//configuration:
#include <LiquidCrystal.h>
const int rs = 6, e = 7, d4 = 8, d5 = 9, d6 = 10, d7 = 11;
LiquidCrystal lcd(rs, e, d4, d5, d6, d7);
String message = "";

//setup:
void setup() {
  lcd.begin(16,2);
} void loop() {
  lcd.setCursor (0,0);

  // printing a message
  lcd.println(message);
  lcd.blink();
```



Software Components



Code Snippet 1: Morse Code Inputs

```
//For letters
char* letters[] = {
    ".-", "-...", "-.-.", "-..", ". ", "..-.", "---.", "....", "..", // A-I
    "----", "-.-", "...", "--", "-.", "----", "...", "--.-", "-.", // J-R
    "...", "-", "-.-", "...-", ".--", "-..-", "-.--", "--.." // S-Z
};

//For Numbers
char* numbers[] = {
    "-----", ".-----", "..-----", "...--", "....-", ".....",
    "-.....", "--...", "---..", "----."
};
```

Code Snippet 2: Translating the button press duration to Morse code

```
if (switchDuration < dotLength) {  
    morseCode += "."; // dot if button was pressed for a  
    shorter time than dotLength  
} else {  
    morseCode += "-";  
}  
Serial.println(morseCode); //prints current morseCode  
string  
}
```

Code Snippet 3: Checking for a button press to trigger message translation

```
buttonState = digitalRead(buttonPin);  
switchState = digitalRead(SW_pin);
```


Code Snippet 4: If the button is pressed and the switch is not, start translation

```
if (buttonState == HIGH && switchState == HIGH && morseCode.length() > 0) {  
    translateMessage(message, morseCode); // translate morseCode into message  
    buttonStartTime = millis();           // record start time  
    digitalWrite(LEDpin, HIGH);          // turn on built-in LED  
    delay(50);  
    while (buttonState == HIGH) {         // while switch is pressed  
        buttonState = digitalRead(buttonPin); // Update buttonState within the loop  
        switchState = digitalRead(SW_pin);  
    }  
    delay(50);  
    digitalWrite(LEDpin, LOW); // turn off built-in LED  
    buttonDuration = millis() - buttonStartTime;  
    if (buttonDuration > dotLength) {  
        message += "_"; // add a space if button was held down for 1/2 second or more  
    }  
}
```

Code Snippet 5: takes message and morseCode as inputs

```
String translateMessage(String& message, String& morseCode) {  
    // uses & identifier so the global variable is changed in the function  
    if (morseCode.length() > 0) {  
        char translatedChar = morseToChar(morseCode);  
        morseCode = ""; // resets morseCode for the next character  
        if (translatedChar != '\0') {  
            message += translatedChar; // add the translated character to the  
message  
        }  
    }  
    return message;  
}
```

Software Struggles

- Translation
 - Displaying according to input
 - Differentiating new word and character (space)
 - Using pass by reference to modify global variables
 - TIMING became a factor: understanding how to use the millis function, arrays

Duration of Morse Code

Presses:

```
int dotLength = 500
```

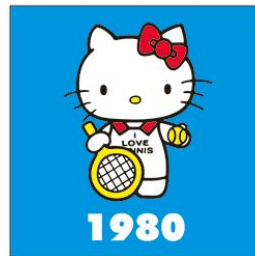
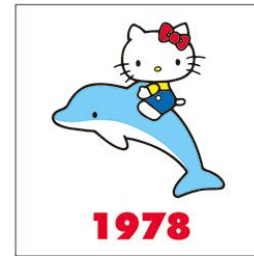
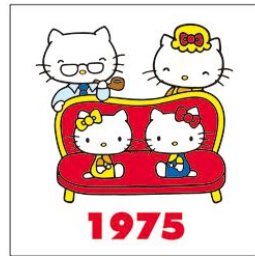


International Morse Code

- 1 dash = 3 dots.
- The space between parts of the same letter = 1 dot.
- The space between letters = 3 dots.
- The space between words = 7 dots.

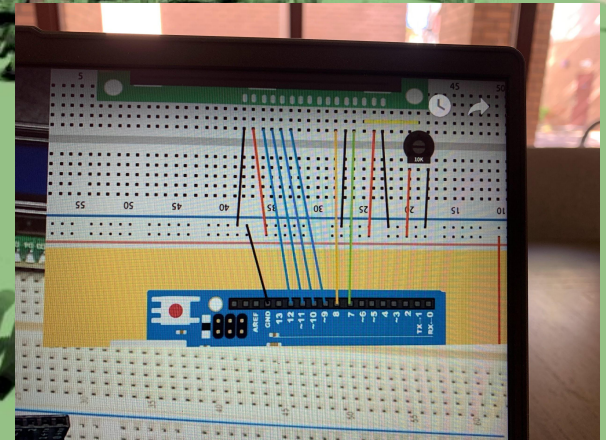
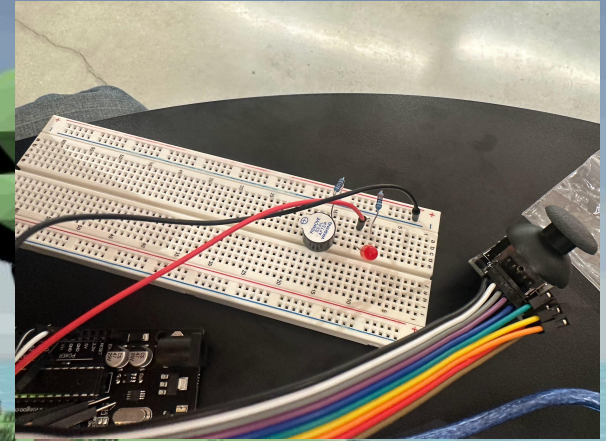
A	● —	V	● ● ● —
B	— ● ● ●	W	— — —
C	— ● — ●	X	— ● ● —
D	— ● ●	Y	— ● — —
E	●	Z	— — ● ●
F	● ● — ●	.	● — ● — ● —
G	— — ●	,	— — ● ● — —
H	● ● ● ●	?	● — — — ● ●
I	● ●	/	— ● ● — ●
J	● — — — —	@	— — — ● — ●
K	— ● —	1	● — — — —
L	● — ● ●	2	● ● — — —
M	— —	3	● ● ● — —
N	— ●	4	● ● ● ● —
O	— — —	5	● ● ● ● ●
P	● — — ●	6	— ● ● ● ●
Q	— — ● —	7	— — — ● ●
R	● — ●	8	— — — — ●
S	● ● ●	9	— — — — ●
T	—	0	— — — — —
U	● ● —		

Check-In Timeline



Check In #1 (November 21st):

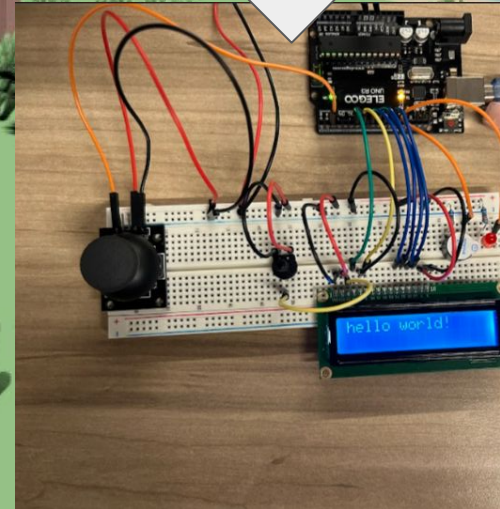
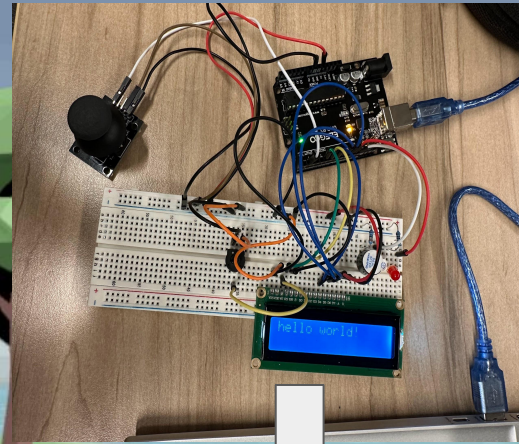
- hardware components working in compatibility with each other (except LCD monitor) ✓
- **in progress:** inputting morse code with joystick
- **in progress:** getting LCD to function with other components



Check In #2 (November 29th):

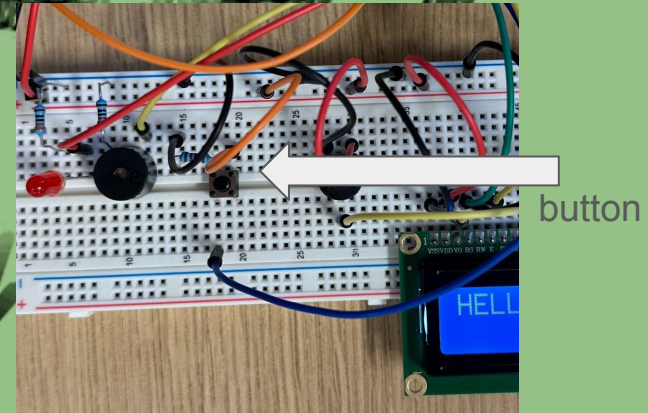
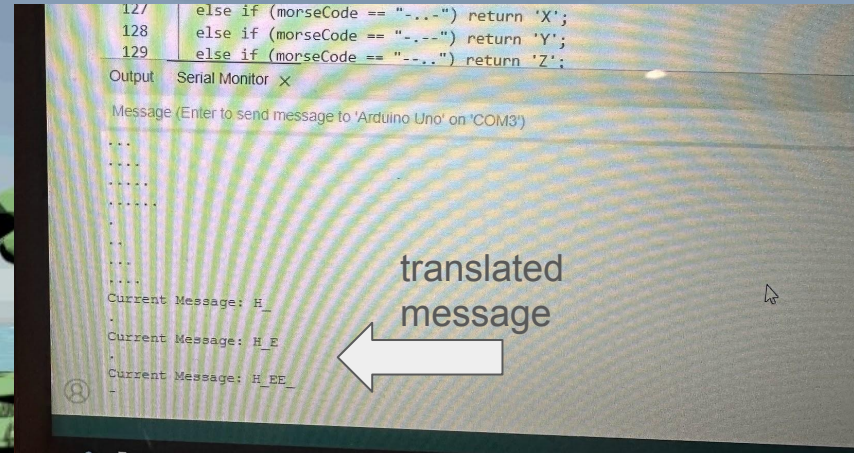
- circuit setup finalized ✓
- LCD monitor printing with duration of button ✓
- morse code language established in Arduino code ✓
- **in progress:** translating morse code characters into letters, displaying on LCD monitor

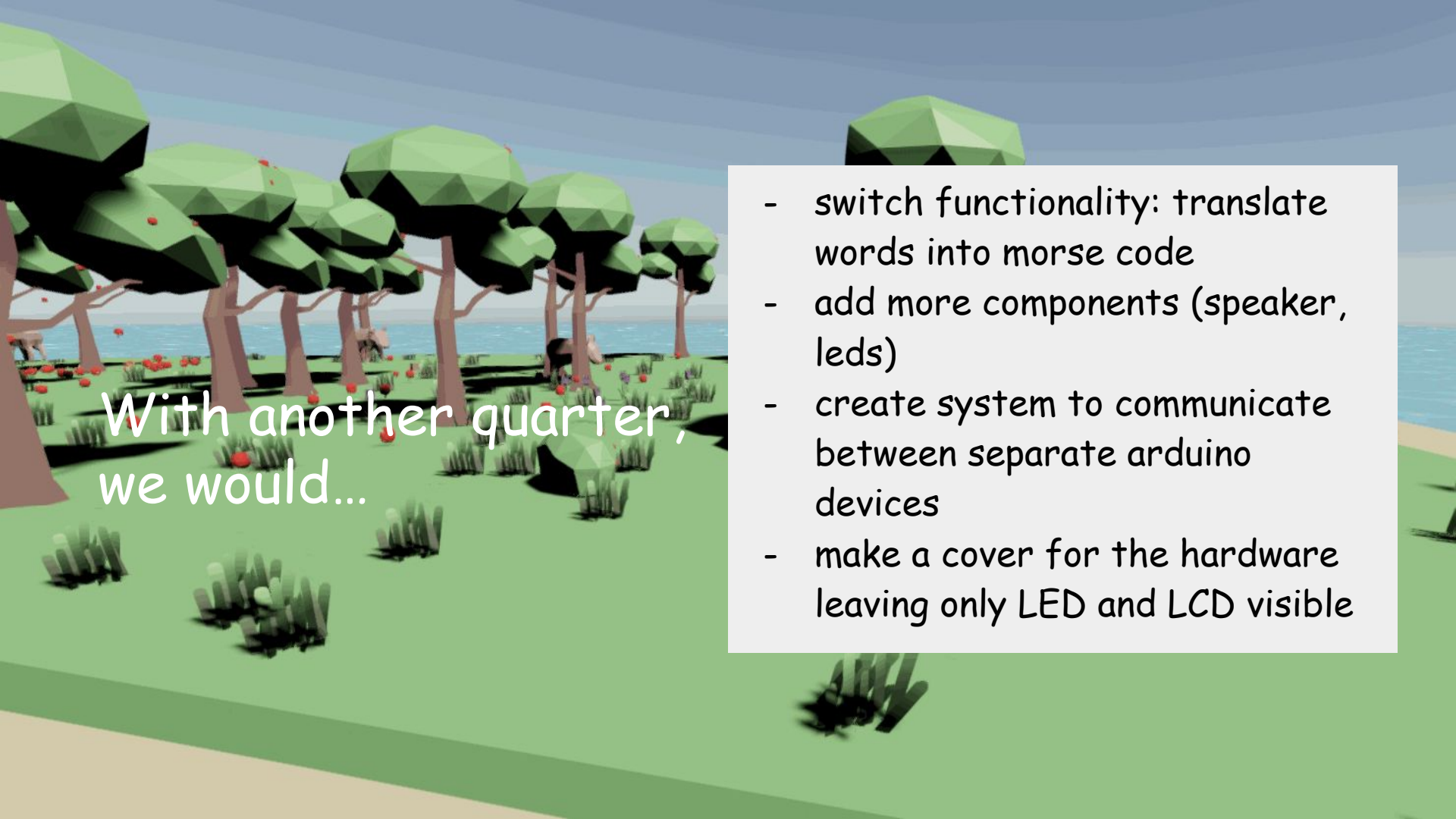
```
if (morseCode == ".-") return 'A';  
else if (morseCode == "-...") return 'B';  
else if (morseCode == "-.-.") return 'C';  
else if (morseCode == "-..") return 'D';
```



Check In #3 (December 4th):

- code input translating on serial monitor ✓
- according characters printing on LCD monitor ✓
- fixing bugs, other technical issues
- **issue:** no space indication
- **solution:** added button component to manually differentiate words





With another quarter,
we would...

- switch functionality: translate words into morse code
- add more components (speaker, leds)
- create system to communicate between separate arduino devices
- make a cover for the hardware leaving only LED and LCD visible

Questions?

MORSE CODE!!



A white cat is lying on its side on a red blanket outdoors. The background shows a brick wall and some greenery. The cat's head is turned towards the left, and its body is stretched out.

thank you

